Transactions: Propagation and Isolation in Spring

Spring's `@Transactional` annotation defines how a method participates in a transaction and how that transaction should behave.

```
@Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        timeout = 5,
        readOnly = false,
        rollbackFor = IOException.class
)
```

The most important attributes are:

Attribute	Responsibility
`propagation`	Decides whether the method joins, creates, suspends, or rejects a transaction
`isolation`	Controls how concurrent transactions can observe one another's changes
`timeout`	Sets the maximum time allowed for the transaction
`readOnly`	Provides a hint that the transaction will only read data
`rollbackFor`	Adds exception types that should cause rollback

> `@Transactional` is normally applied through a Spring-generated proxy. Therefore, the method must be invoked through that proxy for transactional interception to occur.

Default `@Transactional` Behaviour

Writing:

```
@Transactional
public void performOperation() {
    // Business operations
}
```

uses the following defaults:

Setting	Default
Propagation	`Propagation.REQUIRED`
Isolation	`Isolation.DEFAULT`
Read-only	`false`
Timeout	Default of the underlying transaction infrastructure
Rollback	`RuntimeException` and `Error`

Default Propagation: `REQUIRED`

```
If a transaction exists → join it
If no transaction exists → create one
```

Default Isolation: `DEFAULT`

`Isolation.DEFAULT` tells Spring to use the default isolation level of the underlying transaction infrastructure, which is commonly determined by the database.

Default Read-Write Behaviour

```
readOnly = false
```

The transaction is treated as a normal read-write transaction.

Default Timeout

Spring uses the default timeout of the transaction infrastructure. This does not mean that a timeout can never occur. Related timeout rules may still come from

the transaction manager, database, connection pool, JDBC driver, query, or lock acquisition settings.

Commit and Rollback Rules

When Does a Transaction Commit?

A transaction normally commits when all three conditions are satisfied:

1. The transactional method completes normally.
2. The transaction has not been marked as rollback-only.
3. The actual commit operation succeeds.

Return values such as the following do not represent transaction failure:

```
false
null
Optional.empty()
ResponseEntity.badRequest()
```

They are ordinary Java return values. Spring does not inspect their business meaning to decide whether it should roll back.

Rollback for Runtime Exceptions

By default, an unchecked exception causes rollback when it escapes the transactional method.

```
@Transactional
public void transfer() {
    debit();
    throw new IllegalStateException("Transfer failed");
}
```

```
RuntimeException escapes the method
                ↓
```

```
Default rollback rule matches
                ↓
Transaction rolls back
```

Common examples include:

- `IllegalArgumentException`
- `IllegalStateException`
- `NullPointerException`
- `DataIntegrityViolationException`
- Custom subclasses of `RuntimeException`

Rollback for `Error`

An `Error` also triggers rollback by default. Examples include `OutOfMemoryError`, `StackOverflowError`, `AssertionError`, and `LinkageError`.

An `Error` usually represents a serious JVM or environment failure rather than an ordinary business problem, but it still matches Spring's default rollback policy.

Checked Exception Behaviour

A checked exception does not trigger rollback under Spring's traditional default policy.

```java
@Transactional
public void transfer() throws TransferException {
    debit();
    throw new TransferException("Transfer failed");
}
```

```java
public class TransferException extends Exception {
}
```

The default flow is:

```
Checked exception escapes the method
                ↓
Default rollback rule does not match
                ↓
Spring proceeds towards commit
```

This does not mean that checked exceptions are harmless. It only describes the default rollback rule.

Configuring `rollbackFor`

Use `rollbackFor` when a checked exception should cause rollback:

```
@Transactional(rollbackFor = TransferException.class)
public void transfer() throws TransferException {
    debit();
    throw new TransferException("Transfer failed");
}
```

Multiple exception types can be configured:

```
@Transactional(
        rollbackFor = {
                TransferException.class,
                IOException.class
        }
)
public void transfer() {
    // Business operations
}
```

Transaction Propagation

Propagation answers one central question:

> What should this transactional method do when it is called with or without an existing transaction?

Propagation Modes at a Glance

Propagation mode	If a transaction exists	If no transaction exists
`REQUIRED`	Join it	Create a transaction
`REQUIRES_NEW`	Suspend it and create a new transaction	Create a transaction
`SUPPORTS`	Join it	Run without a transaction
`MANDATORY`	Join it	Throw an exception
`NOT_SUPPORTED`	Suspend it and run without a transaction	Run without a transaction
`NEVER`	Throw an exception	Run without a transaction
`NESTED`	Create a savepoint inside it	Create a transaction

Transaction Suspension

Suspending a transaction does not temporarily commit it. It means that Spring pauses its association with the current transaction and its resources while another scope executes.

The suspended transaction has neither committed nor rolled back. It resumes after the inner scope completes.

Propagation modes that may suspend an existing transaction are:

- `REQUIRES_NEW`: suspends the outer transaction and starts a new transaction.
- `NOT_SUPPORTED`: suspends the outer transaction and runs without a transaction.

`Propagation.REQUIRED`

`REQUIRED` is Spring's default propagation mode.

```
@Transactional
public void placeOrder() {
}
```

is equivalent to:

```
@Transactional(propagation = Propagation.REQUIRED)
public void placeOrder() {
}
```

Its rule is simple:

```
If a transaction exists → join it
If no transaction exists → create one
```

Joining an Existing Transaction

Consider an outer method that calls three transactional services:

```
@Transactional
public void checkout() {
    orderService.createOrder();
    inventoryService.reserveStock();
    paymentService.createPaymentRecord();
}
```

If all three inner methods use `REQUIRED`, they participate in the same physical transaction:

```
Transaction A
├── createOrder()
├── reserveStock()
└── createPaymentRecord()
```

They normally share the same:

- Physical database transaction
- JDBC connection
- Transaction-bound `EntityManager`

- Persistence context
- Final commit or rollback decision

The inner methods create separate logical transaction scopes, but they do not create independent physical transactions.

Inner Transaction Settings

Suppose the outer method declares:

```
@Transactional(
        isolation = Isolation.READ_COMMITTED,
        timeout = 30
)
public void checkout() {
    inventoryService.reserveStock();
}
```

The inner method declares:

```
@Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.SERIALIZABLE,
        timeout = 5
)
public void reserveStock() {
}
```

The inner method joins an already-running physical transaction. It therefore cannot normally replace that transaction's characteristics.

The effective physical transaction continues with the outer settings:

```
Isolation → READ_COMMITTED
Timeout   → 30 seconds
```

Outer Failure After Inner Success

```
@Transactional
public void placeOrder() {
    orderRepository.save(order);
    inventoryService.reserveStock();

    throw new IllegalStateException(
            "Failure after inventory reservation"
    );
}
```

Even if `reserveStock()` completes successfully, both operations roll back because they belong to the same physical transaction. An inner `REQUIRED` method cannot commit independently.

`UnexpectedRollbackException`

A subtle problem occurs when an inner `REQUIRED` method fails but the outer method catches the exception:

```
@Transactional
public void placeOrder() {
    orderRepository.save(new Order());

    try {
        inventoryService.reserveStock();
    } catch (InsufficientStockException exception) {
        System.out.println("Stock reservation failed");
    }

    System.out.println("Order method completed normally");
}
```

The execution flow is:

```
1. The outer method starts Transaction A.
2. reserveStock() joins Transaction A.
3. reserveStock() throws a RuntimeException.
4. Its transactional interceptor marks Transaction A as rollback-only.
5. The outer method catches the Java exception.
6. The outer method completes normally and attempts to commit.
7. Transaction A cannot commit because it is already rollback-only.
8. Spring rolls it back and throws UnexpectedRollbackException.
```

Catching the original exception prevents it from propagating further in Java. It does not remove the rollback-only state already placed on the shared transaction.

Demo: One Shared Transaction

```java
@Transactional
public void outer() {
    firstRepository.save(...);
    innerService.required();
    throw new RuntimeException();
}
```

```java
@Transactional
public void required() {
    secondRepository.save(...);
}
```

Both inserts roll back because both methods participate in the same transaction.

When to Use `REQUIRED`

`REQUIRED` is appropriate when all participating operations represent one business unit of work and must share one final outcome.

Typical examples include:

- Placing an order
- Transferring money

- Registering a student
- Creating an invoice
- Updating employee information
- Reserving inventory
- Cancelling a booking

`Propagation.REQUIRES_NEW`

`REQUIRES_NEW` always executes the method inside a new physical transaction.

```
If an outer transaction exists:
    Suspend it

Create a separate transaction

After the inner transaction completes:
    Resume the outer transaction
```

Execution Flow

```java
@Service
@RequiredArgsConstructor
public class OrderService {

    private final AuditService auditService;

    @Transactional
    public void placeOrder() {
        // Transaction A
        auditService.recordAttempt();
        // Transaction A resumes
    }
}
```

```java
@Service
@RequiredArgsConstructor
public class AuditService {

    private final AuditRepository auditRepository;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void recordAttempt() {
        auditRepository.save(new AuditRecord());
    }
}
```

The flow is:

```
1. placeOrder() starts Transaction A.
2. recordAttempt() is invoked through the AuditService proxy.
3. Spring suspends Transaction A.
4. Spring starts Transaction B.
5. recordAttempt() executes inside Transaction B.
6. Transaction B commits or rolls back.
7. Spring releases Transaction B's resources.
8. Spring resumes Transaction A.
```

Separate Physical Resources

In a typical single-database JPA application, the transactions use separate resources:

```
Outer Transaction A
├── EntityManager A
├── Persistence Context A
└── JDBC Connection A

Inner Transaction B
├── EntityManager B
├── Persistence Context B
└── JDBC Connection B
```

Because the transactions are independent, the inner transaction may define its own isolation level, timeout, read-only status, and rollback rules.

Independent Commit and Rollback

Inner Commits, Outer Rolls Back

```
@Transactional
public void outer() {
    orderRepository.save(...);
    auditService.saveAudit();

    throw new RuntimeException();
}
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAudit() {
    auditRepository.save(...);
}
```

Expected result:

```
Audit transaction → commits
Order transaction → rolls back
```

The audit remains committed because it completed in a separate physical transaction.

Inner Rolls Back, Outer Continues

```
@Transactional
public void outer() {
    orderRepository.save(...);

    try {
```

```
            auditService.saveAuditAndFail();
        } catch (RuntimeException exception) {
            System.out.println("Audit failed");
        }

        order.markCompleted();
    }
```

The inner transaction can roll back without marking the outer transaction as rollback-only. If the exception is caught and the outer transaction remains valid, the outer transaction can still commit.

If the exception is not caught, it reaches the outer transactional boundary. The outer transaction may then roll back because it received a `RuntimeException`, not because the two physical transactions share rollback state.

Visibility of Outer Uncommitted Data

Suppose Transaction A inserts a new order but has not committed it:

```
Transaction A:
INSERT order id = 101
Status: uncommitted
```

Transaction B starts through `REQUIRES_NEW` and attempts to load that order:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void recordOrderAudit(Long orderId) {
    Order order = orderRepository.findById(orderId)
            .orElseThrow();
}
```

Transaction B generally cannot assume that it can see Transaction A's uncommitted insert. An independent transaction should not depend casually on uncommitted state owned by the suspended outer transaction.

Common Use Case: Audit Logging

`REQUIRES_NEW` is commonly used when an audit record must be preserved even if the main business transaction fails. It should be used deliberately because it needs an additional physical transaction and typically another database connection while the outer resources remain suspended.

Other Propagation Modes

`Propagation.SUPPORTS`

```
If a transaction exists → join it
If no transaction exists → run without a transaction
```

`SUPPORTS` means that a transaction is welcome but not required.

It can be suitable for:

- Read helper methods
- Calculations that may use transaction-bound entities
- Methods called both inside and outside larger workflows
- Infrastructure helpers that do not require atomic writes

Its behaviour can change depending on the caller, so it should not be used for a write operation that requires a guaranteed transactional boundary.

`Propagation.MANDATORY`

```
If a transaction exists → join it
If no transaction exists → throw an exception
```

`MANDATORY` creates a clear contract: the caller must establish the transaction.

For example, stock reservation may be designed only as a building block of a larger operation such as placing an order or replacing an order item. It should never execute independently.

```java
@Transactional(propagation = Propagation.MANDATORY)
public void reserveStock() {
}
```

Calling this method without an active transaction causes an exception. Calling it from a transactional service makes it join the existing transaction.

`MANDATORY` versus `REQUIRED`

```
REQUIRED  → join an existing transaction or create one
MANDATORY → join an existing transaction or fail
```

Use `REQUIRED` when the method can represent an independent business operation. Use `MANDATORY` when it should only be part of a transaction defined by its caller.

`Propagation.NOT_SUPPORTED`

```
If a transaction exists → suspend it and run without a transaction
If no transaction exists → run without a transaction
```

The method always executes non-transactionally.

`Propagation.NEVER`

```
If a transaction exists → throw an exception
If no transaction exists → run without a transaction
```

`NEVER` establishes the opposite contract to `MANDATORY`: the method must not execute inside a transaction.

`Propagation.NESTED`

`NESTED` normally does not create a second independent physical transaction. It uses the existing physical transaction and creates a database savepoint.

Savepoint

Suppose Transaction A has completed two operations:

```
Operation 1
Operation 2
SAVEPOINT S1
Operation 3
Operation 4
```

If a failure occurs after `S1`, the transaction can execute:

```
ROLLBACK TO SAVEPOINT S1;
```

This discards Operations 3 and 4 while retaining Operations 1 and 2. The outer transaction remains active and still owns the final commit or rollback decision.

If no outer transaction exists, `NESTED` behaves like `REQUIRED` and creates a new transaction because there is no existing transaction in which to create a savepoint.

> `NESTED` requires support from the chosen transaction manager and underlying resource. It is commonly associated with JDBC savepoints and should not be assumed to work identically in every JPA configuration.

Transaction Isolation

Propagation controls the relationship between transactional method calls. Isolation controls the relationship between transactions executing concurrently.

Concurrent execution is unavoidable because many users may place orders, transfer money, book seats, update inventory, and read reports at the same time.

The database must balance two competing objectives:

```
Correctness ↔ Concurrency
```

Stronger isolation offers more protection but can increase blocking, retries, deadlocks, and resource usage. Weaker isolation allows more concurrency but exposes the application to more anomalies.

Concurrency Problems

Dirty Read

A dirty read occurs when one transaction reads a change made by another transaction that has not committed.

Initial state:

```
Account A = ₹10,000
```

Time	Transaction T1	Transaction T2
1	Begins	
2	Updates balance to ₹2,000	
3		Begins
4		Reads balance as ₹2,000
5	Rolls back	
6	Balance returns to ₹10,000	Continues using ₹2,000

T2 used a value that never became part of the committed database state.

Non-Repeatable Read

A non-repeatable read occurs when a transaction reads the same row twice and receives different committed values.

Initial state:

```
Account A = ₹10,000
```

Time	Transaction T1	Transaction T2
1	Begins	
2	Reads balance as ₹10,000	
3		Begins

Time	Transaction T1	Transaction T2
4		Updates balance to ₹8,000
5		Commits
6	Reads the same row again	
7	Receives ₹8,000	

The second value is not dirty because T2 committed it. The problem is that the same row did not produce a repeatable value inside T1.

Phantom Read

A phantom read concerns a set of rows selected by a condition rather than a change to one previously read row.

T1 runs:

```
SELECT *
FROM employee
WHERE salary > 100000;
```

It receives three employees. T2 then inserts a new matching row and commits:

```
INSERT INTO employee(name, salary)
VALUES ('Employee D', 120000);
```

When T1 repeats the original query, it receives four employees. The additional matching row is the phantom.

Time	Transaction T1	Transaction T2
1	Begins	
2	Finds 3 matching employees	
3		Inserts a matching employee
4		Commits
5	Repeats the query	
6	Finds 4 matching employees	

How Databases Implement Isolation

Databases do not necessarily lock an entire table for every transaction. Depending on the database and isolation level, they may use:

- Row and range locks
- Shared and exclusive locks
- Multi-Version Concurrency Control (MVCC)
- Consistent snapshots
- Predicate locking
- Conflict detection
- Transaction aborts and retries

The same isolation level may therefore have different performance characteristics and edge-case behaviour across database systems.

Spring Isolation Levels

Spring provides the following options:

```
Isolation.DEFAULT
Isolation.READ_UNCOMMITTED
Isolation.READ_COMMITTED
Isolation.REPEATABLE_READ
Isolation.SERIALIZABLE
```

`Isolation.DEFAULT`

```
@Transactional(isolation = Isolation.DEFAULT)
public void performOperation() {
}
```

Spring does not request a different isolation level. It uses the default of the underlying transaction infrastructure.

Common database defaults include:

```
MySQL InnoDB → REPEATABLE READ
PostgreSQL   → READ COMMITTED
```

`DEFAULT` is not an additional database isolation level created by Spring.

`Isolation.READ_UNCOMMITTED`

```java
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void generateReport() {
}
```

This is the weakest standard isolation level. A transaction may observe another transaction's uncommitted changes.

```
Dirty read          → Possible
Non-repeatable read → Possible
Phantom read        → Possible
```

It is rarely suitable for business operations because decisions may be based on data that is later rolled back.

`Isolation.READ_COMMITTED`

```java
@Transactional(isolation = Isolation.READ_COMMITTED)
public void performOperation() {
}
```

A transaction cannot read another transaction's uncommitted changes.

```
Dirty read          → Prevented
Non-repeatable read → Possible
Phantom read        → Possible
```

Each statement may see the latest data committed before that statement begins, so two reads inside one transaction can still produce different results.

`Isolation.REPEATABLE_READ`

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void performOperation() {
}
```

In the standard isolation model:

```
Dirty read          → Prevented
Non-repeatable read → Prevented
Phantom read        → May occur
```

Actual behaviour depends on the database implementation. For example, an MVCC-based database may prevent some phantom-read scenarios even though the SQL standard permits them at this level.

`Isolation.SERIALIZABLE`

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void performCriticalOperation() {
}
```

`SERIALIZABLE` is the strongest standard isolation level. Concurrent transactions must produce a result equivalent to some valid serial order.

For two transactions, the result should be equivalent to either:

```
T1 completes, then T2 completes
```

or:

```
T2 completes, then T1 completes
```

The transactions may still overlap physically. The database can enforce serializable behaviour using locks, MVCC, predicate or range locks, conflict detection, or transaction abortion and retry.

```
Dirty read            → Prevented
Non-repeatable read   → Prevented
Phantom read          → Prevented
Serialization anomaly → Prevented
```

Isolation-Level Comparison

Isolation level	Dirty read	Non-repeatable read	Phantom read
`READ_UNCOMMITTED`	Possible	Possible	Possible
`READ_COMMITTED`	Prevented	Possible	Possible
`REPEATABLE_READ`	Prevented	Prevented	Standard permits it; the actual database may prevent it
`SERIALIZABLE`	Prevented	Prevented	Prevented

Choosing the Right Isolation Level

Do not choose the strongest isolation level automatically. Choose the weakest level that still preserves the correctness rules of the business operation.

Consider:

- What incorrect outcome can occur if data changes during the transaction?
- Does the method read the same row or query result more than once?
- Can two users update the same business data concurrently?
- Can the operation tolerate a retry?
- What are the database's actual guarantees at the chosen level?
- Would optimistic or pessimistic locking solve the specific problem more precisely?

Final Summary

Propagation

- `REQUIRED` joins an existing transaction or creates one.
- `REQUIRES_NEW` creates an independent physical transaction and suspends the outer one.
- `SUPPORTS` uses a transaction only when one already exists.
- `MANDATORY` requires the caller to provide a transaction.
- `NOT_SUPPORTED` always runs without a transaction.
- `NEVER` rejects execution inside a transaction.
- `NESTED` uses a savepoint inside an existing transaction when supported.

Isolation

- Isolation controls what concurrent transactions can observe.
- Dirty reads expose uncommitted data.
- Non-repeatable reads change the value of the same row between reads.
- Phantom reads change the set of rows returned by the same condition.
- Stronger isolation improves protection but can reduce concurrency.
- Actual behaviour depends on both the selected isolation level and the database implementation.

Rollback

- Normal completion generally leads to commit.
- `RuntimeException` and `Error` trigger rollback by default.
- Checked exceptions do not trigger rollback by default.
- Use `rollbackFor` when a checked exception must roll back the transaction.

- Catching an exception does not clear a rollback-only marker already placed on a shared transaction.